

Agentic AI HiLabs Hackathon

Ayush Kumar Mandal

Enrollment No: 23116020

Indian Institute of Technology Roorkee

Priyani Nagle

Enrollment No: 23125026

Indian Institute of Technology Roorkee

March 14, 2026

1 Abstract

Modern healthcare data systems rely on complex provider roster pipelines that ingest, validate, transform, and distribute provider data across multiple downstream systems. Failures or inefficiencies in these pipelines can lead to delays in data availability, operational disruptions, and degraded analytics quality.

Traditional monitoring solutions rely primarily on dashboards and manual log analysis, which provide visibility but lack diagnostic reasoning capabilities. As pipeline complexity grows, identifying root causes of failures becomes increasingly challenging.

This project presents **RosterIQ**, an Agentic AI-driven analytics system designed to diagnose provider roster pipeline issues through natural language queries. The system integrates a multi-agent architecture, three complementary memory systems (episodic, semantic, and procedural), deterministic analytical tools, and interactive visualizations.

RosterIQ enables users to ask questions such as:

- Which organizations cause most pipeline failures?
- Why are pipelines failing?
- Show pipeline bottlenecks.
- Search causes of ETL pipeline bottlenecks.

The system interprets the query, selects an analytical workflow, invokes the appropriate diagnostic tools, retrieves contextual knowledge from memory, and produces insights or visualizations.

This report describes the design methodology, system architecture, memory systems, implementation decisions, innovations, and development challenges encountered while building the system.

2 Introduction

Provider roster pipelines are critical components of healthcare data infrastructure. These pipelines manage the ingestion and processing of provider information such as physician identifiers, affiliations, specialties, and location data.

Typical provider roster pipelines involve multiple stages including:

- ingestion from upstream source systems
- preprocessing and validation
- mapping and transformation
- data enrichment
- downstream publishing

Failures or delays in these stages can originate from multiple factors such as:

- malformed provider data
- schema mismatches between source systems
- unstable upstream data sources
- slow transformation processes
- stage-level processing bottlenecks

Current operational monitoring tools primarily provide dashboards displaying metrics such as failure counts, stage durations, or success rates. While useful for observation, these dashboards do not provide diagnostic reasoning.

Operators must manually investigate logs and metrics to determine why failures occur.

The goal of this project is therefore to design a system that allows operators to interact with pipeline analytics using natural language queries while enabling the system to perform autonomous reasoning over operational data.

To achieve this goal, we implemented **RosterIQ**, an agentic AI analytics assistant capable of interpreting analytical queries, performing pipeline diagnostics, retrieving contextual knowledge, and presenting results through explanations and visualizations.

3 Problem Interpretation and Solution Approach

Most existing pipeline monitoring systems fall into two categories:

1. Static dashboards
2. Log-based troubleshooting tools

Static dashboards provide high-level metrics but lack reasoning capabilities. Log-based tools provide detailed information but require manual analysis.

Therefore, an effective system must support:

1. Understanding natural language queries
2. Selecting analytical workflows
3. Executing deterministic analytical tools
4. Retrieving contextual knowledge
5. Presenting insights through explanations and visualizations

To achieve this, we adopted an **agent-oriented architecture**. The system separates reasoning from execution.

- The LLM performs reasoning and task planning
- Deterministic tools perform analytical computation

This design follows the **ReAct paradigm** introduced by Yao et al. (2023), where language models alternate between reasoning and tool usage.

4 System Overview

RosterIQ is designed as an Agentic AI system where multiple specialized agents collaborate under a Supervisor Agent.

The architecture integrates:

- Analytical tools
- Structured memory systems
- Autonomous decision-making
- Visualization modules
- External knowledge retrieval

The system transforms pipeline monitoring from static dashboards into an intelligent analytics assistant.

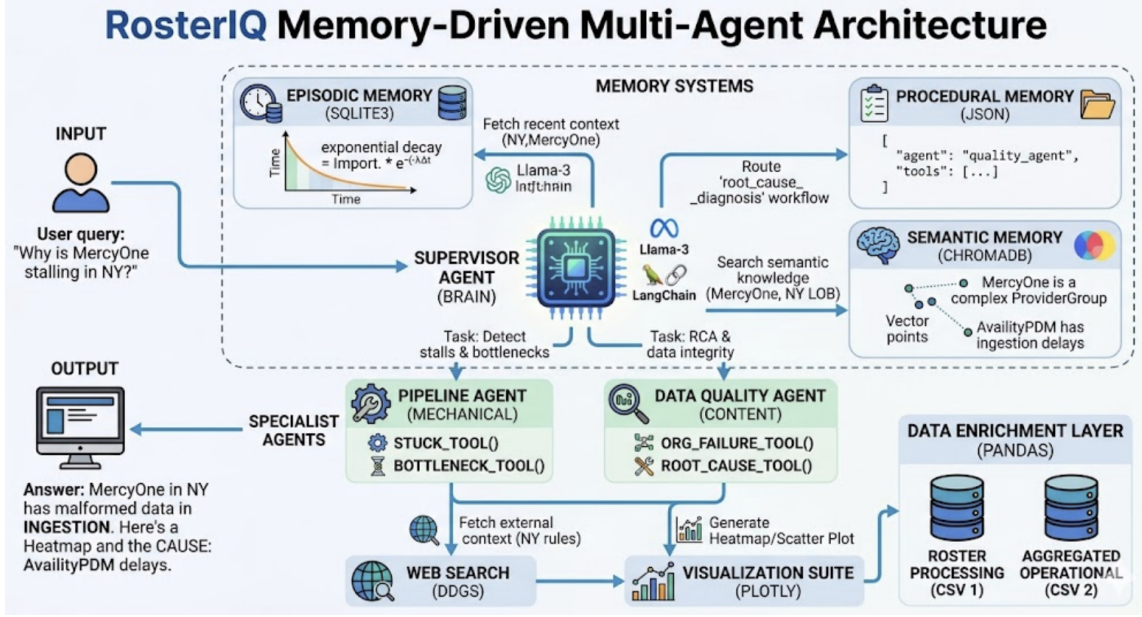


Figure 1: Semantic Memory Retrieval Workflow

4.1 System Architecture

The overall architecture of **RosterIQ** follows a **memory-driven multi-agent design**, where multiple specialized agents collaborate under the coordination of a *Supervisor Agent*. As illustrated in Figure ??, the system integrates analytical tools, structured memory systems, and data processing modules to enable intelligent pipeline diagnostics.

When a user submits a query, the Supervisor Agent first interprets the request and determines the appropriate analytical workflow. During this process, the agent retrieves contextual information from three memory systems: *episodic memory*, *procedural memory*, and *semantic memory*. Episodic memory stores past interactions using a SQLite database, procedural memory defines analytical workflows using JSON-based procedures, and semantic memory retrieves domain knowledge through a vector database implemented with ChromaDB.

Based on the selected procedure, the supervisor routes the query to specialized agents such as the **Pipeline Agent** or the **Data Quality Agent**. The Pipeline Agent focuses on detecting pipeline stalls and performance bottlenecks, while the Data Quality Agent investigates failure patterns related to organizations or source systems. These agents utilize analytical tools that operate on operational datasets loaded through the data enrichment layer.

The system can also retrieve external information through a web search module and generate visual insights using a visualization suite implemented with Plotly. Finally, the generated insights are returned to the user, and the interaction is stored in episodic memory to maintain conversational context for future queries.

This architecture enables RosterIQ to combine data analytics, domain knowledge

retrieval, and memory-driven reasoning to provide intelligent diagnostics for complex provider roster pipeline operations.

4.2 Supervisor Agent

The **Supervisor Agent** serves as the central orchestration component of the system. It receives user queries and determines the appropriate analytical procedure required to generate a response.

Its primary responsibilities include:

- Interpreting natural language queries
- Selecting analytical workflows from procedural memory
- Routing queries to the appropriate specialist agents
- Managing tool invocation and execution

The supervisor analyzes the intent of each query and determines whether it relates to pipeline diagnostics, data quality investigation, visualization generation, or external research tasks.

Based on this interpretation, the supervisor routes the request to the appropriate analytical procedure as shown in Table 1.

Query Type	Procedure
Pipeline diagnostics	<code>pipeline_health_check</code>
Failure analysis	<code>failure_organization_analysis</code>
Root cause analysis	<code>root_cause_diagnosis</code>
Visualization requests	<code>visualization_analysis</code>
External research	<code>external_research</code>

Table 1: Query routing logic used by the Supervisor Agent

4.2.1 Pipeline Health Agent

The **Pipeline Health Agent** is responsible for analyzing the operational performance of the provider roster pipeline. It continuously monitors execution metrics to detect abnormalities and identify potential performance issues within the pipeline.

Its main capabilities include:

- Detecting stuck pipeline operations
- Identifying processing bottlenecks
- Analyzing abnormal stage durations

- Monitoring pipeline health indicators

To perform these analyses, the agent utilizes specialized analytical tools such as:

- `detect_stuck_operations`
- `detect_stage_bottlenecks`

By analyzing pipeline execution metrics and operational logs, the agent can identify anomalies in processing behavior and generate diagnostic insights that assist in maintaining pipeline stability and efficiency.

4.2.2 Data Quality Agent

The **Data Quality Agent** focuses on analyzing failures related to data inconsistencies and ingestion issues within the provider roster pipeline. It investigates data-related anomalies that may disrupt pipeline execution and helps identify the underlying causes of such failures.

Its key responsibilities include:

- Identifying organizations responsible for pipeline failures
- Analyzing source system error patterns
- Performing root cause diagnostics

To perform these analyses, the agent utilizes specialized analytical tools such as:

- `organization_failure_analysis`
- `source_system_failure_analysis`
- `generate_root_cause`

By examining failure logs, error patterns, and source system behavior, the agent can detect systemic issues such as unreliable data sources, schema inconsistencies, or malformed data formats. These insights enable faster troubleshooting and help improve the overall reliability of the pipeline.

4.3 Web Search Tool

The system also integrates an external **Web Search Tool** that allows the agent to retrieve relevant information from the internet. This capability enables the system to incorporate broader technical knowledge beyond the internal dataset.

Through this integration, the system can access information related to topics such as:

- ETL pipeline architectures
- Data engineering best practices
- Pipeline debugging techniques
- Common causes of pipeline bottlenecks

For example, a user may submit the following query:

“Search common causes of ETL pipeline bottlenecks.”

In response, the system retrieves relevant articles, technical documentation, and on-line resources. The retrieved information is then summarized and incorporated into the agent’s analytical response, allowing the system to provide more comprehensive and context-aware insights.

4.4 Memory System Architecture

A key component of the **RosterIQ** system is its **multi-memory architecture**, which enables the AI agent to maintain contextual awareness, apply structured reasoning workflows, and retrieve relevant domain knowledge during analysis.

Inspired by cognitive architectures used in intelligent systems, RosterIQ integrates three complementary memory types:

- Episodic Memory
- Semantic Memory
- Procedural Memory

Each memory type plays a distinct role in the reasoning process. Episodic memory stores past interactions and analytical experiences, semantic memory captures structured domain knowledge, and procedural memory maintains the workflows required to perform analytical tasks.

Together, these memory systems enable the platform to behave as a *context-aware intelligent analytics agent* rather than a simple query-processing tool. By combining historical context, domain knowledge, and analytical procedures, the system can generate more accurate, consistent, and explainable insights.

4.4.1 Episodic Memory

Episodic memory stores historical interactions between the user and the system, allowing the agent to maintain conversational context across multiple queries. This memory enables the system to recall past investigations, previously generated insights, and user queries.

In the **RosterIQ** system, episodic memory is implemented using a **SQLite** database located at:

`memory/episodic_memory.db`

Each interaction stored in the database contains the following information:

Field	Description
user_query	The query submitted by the user
agent_response	The response generated by the system
timestamp	Time when the interaction occurred
importance	A weight representing the importance of the interaction

Table 2: Structure of records stored in episodic memory

An example stored record is shown below:

user_query	agent_response
Which organizations fail most?	Trinity Health contributes the highest number of failures.
Show pipeline bottlenecks	Bottlenecks detected in the DART generation stage.

Table 3: Example episodic memory records

When a new query is received, the system retrieves relevant past interactions using the `fetch_recent()` function. Instead of returning only the most recent entries, the system ranks stored memories using a **time-decay scoring mechanism**.

Time Decay Scoring

Time-Decay Based Memory Retrieval The relevance of each stored interaction is calculated using an **exponential decay function**:

$$Score = Importance \times e^{-\lambda \Delta t} \quad (1)$$

where:

- **Importance** represents the base weight assigned to the memory.

- λ (lambda) controls how quickly the memory relevance decays.
- Δt represents the time difference between the current time and the stored interaction.

This scoring mechanism ensures that recent interactions receive higher priority while older memories gradually lose influence. However, older memories are not removed from the system; instead, their importance decreases over time.

This mechanism is commonly referred to as **soft forgetting**, where historical interactions remain stored but become less influential during the retrieval process.

Episodic memory improves the system by:

- Maintaining conversational continuity
- Allowing the agent to recall previous investigations
- Providing contextual awareness across multiple queries

4.4.2 Semantic Memory

Semantic memory stores domain knowledge about provider roster pipelines and data processing behavior. Unlike episodic memory, which records specific events, semantic memory stores general knowledge that can be reused across multiple analytical queries.

Examples of stored knowledge include:

- Causes of ingestion failures
- Data schema mismatch patterns
- Source system reliability insights
- Pipeline stage dependency relationships

Semantic memory is implemented using a **vector database architecture**. The system uses **Sentence Transformers** to convert textual knowledge into numerical vector embeddings. These embeddings are then stored in **ChromaDB**, which enables efficient similarity-based retrieval of relevant information.

Semantic Memory Workflow The workflow for semantic knowledge retrieval is illustrated below: When a query is processed, the system generates an embedding representation of the user query and performs a vector similarity search to retrieve the most relevant knowledge entries.

For example, if the user asks:

“Why are pipelines failing?”

The system may retrieve knowledge related to:

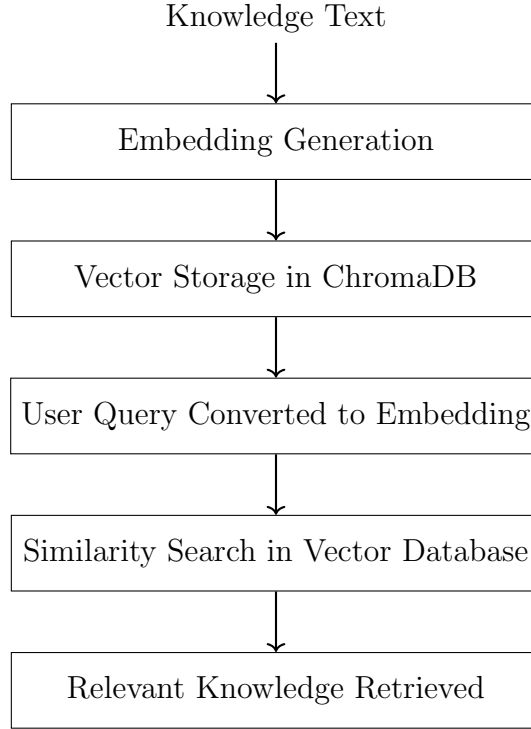


Figure 2: Semantic Memory Retrieval Workflow

- Ingestion failures
- Schema mismatches
- Source system inconsistencies

The retrieved knowledge is then injected into the reasoning context of the agent, improving the accuracy, depth, and explainability of the generated response.

Semantic memory provides several advantages:

- Contextual knowledge retrieval
- Improved reasoning accuracy
- Ability to match queries with semantically similar knowledge
- Domain-aware explanations

4.4.3 Procedural Memory

Procedural memory defines how the system performs different types of analysis. While semantic memory stores knowledge about the domain, procedural memory stores structured workflows that guide the system in solving analytical problems.

In the **RosterIQ** system, procedural memory is implemented using a **JSON-based procedure registry** stored at:

`memory/procedures.json`

Each procedure stored in this registry defines:

- The type of problem it addresses
- The analytical steps required to solve the problem
- The tools or specialist agents that should be invoked

For example, a *pipeline delay analysis* procedure may contain the following steps:

- Identify abnormal stage durations
- Compare execution metrics against historical averages
- Highlight pipeline stages with potential bottlenecks

These procedures guide the **Supervisor Agent** in determining how a user query should be processed.

When a query is received, the Supervisor Agent first analyzes the query intent and then selects the most relevant procedure from procedural memory. The selected procedure determines:

- Which specialist agent should be used
- Which analytical tools should be invoked
- What workflow should be executed

By storing structured workflows in procedural memory, the system ensures that analytical processes remain consistent and reproducible across different queries.

Procedural memory provides several benefits:

- Structured problem-solving workflows
- Consistent analysis strategies
- Modular reasoning architecture
- Efficient task routing across agents

5 Visualization System

The RosterIQ system includes a visualization module that converts analytical results into interactive charts, allowing users to quickly understand pipeline performance and operational issues. While textual responses explain the analytical results, visualizations help reveal patterns, anomalies, and trends within large operational datasets.

The visualization module is implemented using **Plotly Express**, which enables the generation of dynamic and interactive charts. These visualizations are automatically produced based on the user's query and the type of analytical insight requested.

Three main types of visualizations are supported in the system.

5.1 Pipeline Stage Health Heatmap

This visualization presents the health distribution of different pipeline stages such as preprocessing, mapping approval, ISF generation, DART generation, review, validation, and SPS loading.

The system aggregates the health indicators of these stages and displays them in a heatmap representation. This allows operators to quickly identify which stages of the pipeline frequently experience operational issues or instability.

5.2 Duration Anomaly Chart

The duration anomaly chart compares the actual execution duration of a pipeline stage with its historical average duration. The visualization is implemented using a scatter plot in which:

- The x-axis represents the average stage duration.
- The y-axis represents the actual observed duration.

If the observed duration significantly exceeds the historical average, the stage is likely experiencing a bottleneck or abnormal delay.

5.3 Failure Distribution Chart

The failure distribution chart identifies organizations responsible for the highest number of pipeline failures. The system aggregates failure counts by organization and displays the top contributors using a bar chart.

This visualization enables operators to quickly identify problematic organizations or unreliable data providers that frequently contribute to pipeline failures.

5.4 Visualization Routing

To simplify user interaction, the system includes a **visualization routing function** that selects the appropriate chart based on keywords detected in the user query.

For example:

- Queries related to failures generate the *failure distribution chart*.
- Queries about bottlenecks generate the *duration anomaly chart*.
- Queries about stage health produce the *pipeline stage health heatmap*.

By integrating automated visualization with the agent-based analytics framework, RosterIQ enables users to interpret pipeline data more effectively and quickly identify operational issues within the system.

6 Data Processing Pipeline

The system processes operational data from two datasets:

6.1 Roster Processing Details

Contains detailed pipeline execution information including:

- Processing stages
- Execution timestamps
- Failure states
- Source systems
- Organization identifiers

6.2 Aggregated Operational Metrics

Provides higher-level operational statistics such as:

- Market-level success rates
- Temporal performance trends

6.3 Data Loading and Preprocessing

The system uses a dedicated **DataLoader module** to load and preprocess pipeline datasets.

The DataLoader performs several important preprocessing steps:

- Loading CSV datasets into Pandas DataFrames
- Normalizing column names to maintain consistency
- Converting timestamp columns into datetime format
- Generating derived analytical metrics

Additional derived features include:

- Total pipeline execution duration
- Stage duration anomaly ratios
- Flags for slow pipeline executions

These derived features allow the system to perform deeper analysis of pipeline performance and identify abnormal behavior.

6.4 Pipeline Intelligence and Diagnostics

The system includes several analytical functions used by agents to investigate pipeline behavior.

6.5 Stuck Pipeline Detection

The system identifies pipeline operations that are stuck by analyzing the **IS_STUCK** flag. These records indicate jobs that have stopped progressing through pipeline stages.

This analysis returns information such as the roster operation ID, organization name, state, source system, and the stage where processing stopped.

6.6 Pipeline Bottleneck Detection

The system detects performance bottlenecks by comparing the actual duration of pipeline stages with their historical average durations. If the duration exceeds the expected value significantly, the stage is flagged as a potential bottleneck.

6.7 Organization Failure Analysis

Failures are grouped by organization to identify provider organizations responsible for the highest number of pipeline errors. This helps detect problematic data providers.

6.8 Source System Failure Analysis

The system also analyzes failures by source system to determine which upstream systems contribute the most operational issues.

6.9 Pipeline Health Summary

Pipeline health indicators across stages are aggregated to provide a summary of operational health. This allows operators to identify stages experiencing frequent warnings or failures.

6.10 Root Cause Analysis

The Root Cause Analysis module identifies the major contributors to pipeline failures.

The system analyzes failed pipeline executions and examines three primary dimensions:

- Organizations responsible for failures
- Pipeline stages where failures occur most frequently
- Source systems contributing to failures

By analyzing these factors simultaneously, the system can identify patterns that reveal the underlying causes of operational issues within the pipeline.

6.11 Insight Generation

The Insight Engine converts analytical results into high-level operational insights.

Instead of returning raw metrics, the system summarizes important findings such as:

- Pipeline stages contributing most failures
- Organizations responsible for repeated errors
- Source systems causing operational instability

These insights allow operators to quickly understand the most critical issues affecting pipeline performance.

7 Streamlit Dashboard

The system provides an interactive user interface built using Streamlit.

Features include:

- Pipeline performance monitoring
- Failure analytics
- Bottleneck detection
- Market success trends
- AI-powered query interface

8 Technology Stack

Component	Technology
LLM	Groq LLaMA 3.1
Agent Framework	LangChain
Vector Database	ChromaDB
Embeddings	Sentence Transformers
Database	SQLite
Visualization	Plotly
Dashboard	Streamlit
Search Engine	DuckDuckGo

Table 4: Technology Stack

9 Key Innovations

The RosterIQ system introduces several innovations that improve the effectiveness of pipeline analytics:

- **Agentic Intelligence:** The system performs autonomous reasoning using specialized agents that interpret user queries, select analytical workflows, and invoke deterministic tools to generate insights.
- **Multi-Memory Architecture:** The integration of episodic, semantic, and procedural memory enables the system to maintain contextual awareness, retrieve domain knowledge, and apply structured analytical procedures.
- **Context-Aware Analytics:** The system combines pipeline execution data, domain knowledge, and historical interactions to produce more informed and explainable analytical insights.

- **Natural Language Interaction:** Users can explore operational insights and diagnose pipeline behavior using intuitive natural language queries instead of manual dashboard exploration or log analysis.

10 Limitations

Current limitations include:

- Limited semantic knowledge base
- Text-based external search
- No automated knowledge learning

11 Future Work

Future improvements may include:

- Automated knowledge learning
- Real-time pipeline monitoring
- Predictive failure detection
- Integration with enterprise observability platforms

12 Conclusion

RosterIQ demonstrates how **agentic AI systems can transform operational analytics**.

By integrating multi-agent reasoning, semantic knowledge retrieval, episodic context memory, and intelligent visualization, the system provides a powerful framework for diagnosing complex pipeline behaviors.

The architecture is scalable and can be extended to domains such as:

- Data platform monitoring
- Cloud infrastructure analytics
- Enterprise observability systems

RosterIQ represents a step toward **self-diagnosing intelligent data infrastructure**.

Github Link : <https://github.com/Priyani-dsai/RosterIQ>